Software Engineering Project May - 2026

Milestone 3 Deliverables for Neighbourly

Submitted By

Team Name: BugsBunny

Team Code: Team-015



IITM Online BS Degree Program,
Indian Institute of Technology, Madras, Chennai
Tamil Nadu, India, 600036

Apartment Association Management System

Project Details

Software Name: Neighbourly - Apartment Association Management System

Date of Submission: 02-08-2026

Team Details

Team Name: BugsBunny
Team Code: Team-015

Name	Roll Number	Role *
YASH KARSH	22f3002322	Project Architect
Juhi Parekh	23f2005473	Scrum Master
SAFWAN HUMAYUN	24f1001022	Frontend Developer
SANIYA NAAZ	23f2002592	Backend Developer
Saumya Sarkar	21f3001551	Backend Developer/Software Tester

*These are the team's primary roles, and members will collaborate across tasks as the project continues.

API Documentation Report

The backend APIs developed for Neighbourly are documented using a Swagger-compatible OpenAPI YAML specification. The documentation provides a complete description of every implemented endpoint, including the supported HTTP methods, endpoint URLs, request and response schemas, user story mapping, expected status codes, and error handling information. This centralized documentation enables seamless communication between the frontend and backend while simplifying testing, debugging, and future development.

Endpoint Execution Distribution

#	API Operation	Expected Status	Actual Status	First Attempt
1	POST /api/v1/auth/register-committee	200	200	PASS FIRST TRY
2	POST /api/v1/auth/signup	200	200	PASS FIRST TRY
3	POST /api/v1/auth/login	200	200	PASS FIRST TRY
4	POST /api/v1/auth/refresh	200	200	PASS FIRST TRY
5	POST /api/v1/auth/logout	200	200	PASS FIRST TRY
6	POST /api/v1/auth/forgot-password	200	200	PASS FIRST TRY
7	POST /api/v1/auth/reset-password	200	200	PASS FIRST TRY
8	GET /api/v1/auth/me	200	200	PASS FIRST TRY
9	GET /api/v1/auth/pending-verifications	200	200	PASS FIRST TRY
10	POST /api/v1/auth/verify/{user_id}	200	200	FAILED THEN FIXED
11	POST /api/v1/blocks	200	200	PASS FIRST TRY
12	GET /api/v1/blocks	200	200	PASS FIRST TRY
13	GET /api/v1/blocks/{block_id}	200, 404	200, 404	PASS FIRST TRY
14	PATCH /api/v1/blocks/{block_id}	200, 404	200, 404	PASS FIRST TRY
15	DELETE /api/v1/blocks/{block_id}	200, 400, 404	200, 400, 404	PASS FIRST TRY
16	POST /api/v1/flats	200	200	PASS FIRST TRY
17	GET /api/v1/flats	200	200	PASS FIRST TRY
18	GET /api/v1/flats/{flat_id}	200, 404	200, 404	PASS FIRST TRY
19	PATCH /api/v1/flats/{flat_id}	200, 404	200, 404	PASS FIRST TRY
20	POST /api/v1/residents/me/profile-picture	200, 400	200, 400	PASS FIRST TRY
21	DELETE /api/v1/residents/me/profile-picture	200	200	PASS FIRST TRY
22	POST /api/v1/admin/complaint-categories	200, 422	200, 422	PASS FIRST TRY
23	GET /api/v1/admin/complaint-categories	200	200	PASS FIRST TRY
24	POST /api/v1/complaints	200, 403, 422	200, 403, 422	PASS FIRST TRY
25	GET /api/v1/complaints	200	200	PASS FIRST TRY
26	GET /api/v1/complaints/{complaint_id}	200, 404	200, 404	PASS FIRST TRY
27	PATCH /api/v1/complaints/{complaint_id}	200, 400, 404	200, 400, 404	PASS FIRST TRY
28	POST /api/v1/complaints/{complaint_id}/assign	200, 403, 404	200, 403, 404	PASS FIRST TRY
29	POST /api/v1/complaints/{complaint_id}/comments	200, 400, 404	200, 400, 404	PASS FIRST TRY
30	POST /api/v1/admin/facilities	200, 403	200, 403	PASS FIRST TRY
31	GET /api/v1/admin/facilities	200, 403	200, 403	PASS FIRST TRY
32	GET /api/v1/admin/facilities/{facility_id}	200, 403, 404	200, 403, 404	PASS FIRST TRY
33	PATCH /api/v1/admin/facilities/{facility_id}	200, 403, 404	200, 403, 404	PASS FIRST TRY
34	POST /api/v1/facility-bookings	200, 409, 400	200, 409, 400	PASS FIRST TRY
35	GET /api/v1/facility-bookings	200	200	FAILED THEN FIXED

36	GET /api/v1/facility-bookings/{booking_id}	200, 404	200, 404	PASS FIRST TRY
37	PATCH /api/v1/facility-bookings/{booking_id}	200, 403, 404, 409	200, 403, 404, 409	PASS FIRST TRY
38	POST /api/v1/visitors/pre-register	200	200	PASS FIRST TRY
39	GET /api/v1/visitors/requests	200	200	FAILED THEN FIXED
40	POST /api/v1/visitors/{request_id}/verify-otp	200, 400	200, 400	PASS FIRST TRY
41	POST /api/v1/visitors/logs/{log_id}/exit	200	200	PASS FIRST TRY
42	GET /api/v1/visitors/logs	200	200	PASS FIRST TRY
43	GET /health	200	200	PASS FIRST TRY

Endpoint Execution Summary

Metric	Count
Total Endpoints	43
Pass First Try	40
Failed Then Fixed	3
Overall Pass Rate	97.7%

Mismatch Summary

The rows listed below indicate endpoints that returned mismatched states before subsequent bug fixes were deployed.

Case ID	API Endpoint	Expected Output	Actual Output	Root Cause / Difference Summary
MM-VERIFY-001	POST /api/v1/auth/verify/{user_id}	200 OK with occupancy_type: "Rental"	200 OK with occupancy_type: "Owner"	The verify endpoint did not persist the occupancy_type field from the request body. The OccupancyHistory record defaulted to Owner instead of the supplied Rental value.
MM-BOOK-001	GET /api/v1/facility-bookings	200 OK with an empty list []	200 OK with 3 booking records belonging to other residents	The resident booking list endpoint returned bookings from other residents because the list_bookings service did not apply the resident_id filter when resident context was not enforced by the test client.
MM-VIS-001	GET /api/v1/visitors/requests	200 OK with 1 visitor request	200 OK with 2 visitor requests	The endpoint returned a visitor request from a previous test because the seeded_db fixture did not delete VisitorRequest records between test executions.
MM-VIS-002	GET /api/v1/visitors/requests	200 OK with an empty list []	200 OK with 2 visitor request records	Same fixture cleanup issue as MM-VIS-001. The seeded_db fixture failed to purge VisitorRequest records, resulting in cross-test contamination.

Fixed Rows

Case ID	Endpoint	Issue	Fix Applied	Status
MM-VERIFY-001	POST /api/v1/auth/verify/{user_id}	occupancy_type not persisted to OccupancyHistory	Updated verify_user in auth_service.py to pass occupancy_type to move_in service	Fixed
MM-BOOK-001	GET /api/v1/facility-bookings	Test fixture leaked FacilityBooking records	Added FacilityBooking cleanup to seeded_db fixture	Fixed
MM-VIS-001	GET /api/v1/visitors/requests	Test fixture leaked VisitorRequest records	Added VisitorRequest, VisitorLog, and Visitor cleanup to seeded_db fixture	Fixed
MM-VIS-002	GET /api/v1/visitors/requests	Same fixture leakage as MM-VIS-001	Same fix as MM-VIS-001	Fixed

---

API Test Cases

To verify the correctness and reliability of the implemented backend services, comprehensive API testing was carried out using pytest. The test suite validates successful execution, authorization, business logic, input validation, and error handling across the core modules of the Neighbourly system. Each test case records the API endpoint, inputs, expected output, actual output, and the final result, helping identify implementation issues and improve overall software quality.

Test Case 1 - Register Committee Member

Field	Value
API Being Tested	POST /api/v1/auth/register-committee
Inputs	Auth: Admin Bearer token Body: {"email": "new_committee@test.com", "full_name": "New Committee", "phone": "1234567890", "password": "Password123!"}
Expected Output	200 OK; response data contains email: new_committee@test.com, role: Committee, user_id, initial_password
Actual Output	200 OK; response data matches expected fields exactly
Result	Success

```python
async def test_admin_can_create_committee(self, client: AsyncClient, seeded_db: Session):
        _, admin_token = create_test_user(seeded_db, RoleEnum.ADMIN.name, email="admin-committee.test.com")
        response = await client.post(
            "/api/v1/auth/register-committee",
            headers={"Authorization": f"Bearer {admin_token}"},
            json={
                "email": "new_committee@test.com",
                "full_name": "New Committee",
                "phone": "1234567890",
                "password": "Password123!",
            },
        )
        assert response.status_code == status.HTTP_200_OK
        data = response.json()["data"]
        assert data["email"] == "new_committee@test.com"
        assert data["role"] == "Committee"
        assert data["initial_password"] == "Password123!"
        assert "user_id" in data
```

Test Case 2 - Resident User Registration

Field	Value
API Being Tested	POST /api/v1/auth/signup
Inputs	Body: {"email": "newresident@test.com", "password": "Password123!", "full_name": "New Resident", "phone": "1234567890", "role": "Resident"}
Expected Output	200 OK; response data contains email: newresident@test.com, role: Resident, is_verified: false, user_id
Actual Output	200 OK; all expected fields present and correct
Result	Success

```python
async def test_signup_resident(self, client: AsyncClient, seeded_db: Session):
        response = await client.post(
            "/api/v1/auth/signup",
            json={
                "email": "newresident@test.com",
                "password": "Password123!",
                "full_name": "New Resident",
                "phone": "1234567890",
                "role": "Resident",
            },
        )
        assert response.status_code == status.HTTP_200_OK
        data = response.json()["data"]
        assert data["email"] == "newresident@test.com"
        assert data["role"] == "Resident"
        assert data["is_verified"] is False
        assert "user_id" in data
```

Test Case 3 - User Login Authentication

Field	Value
API Being Tested	POST /api/v1/auth/login
Inputs	Body: {"email": "login-success@test.com", "password": "LoginPass123!"}
Expected Output	200 OK; response data contains access_token, refresh_token, token_type: bearer, role: Resident, is_verified: true
Actual Output	200 OK; all expected fields present and correct
Result	Success

```python
async def test_login_success(self, client: AsyncClient, seeded_db: Session):
        email = "login-success@test.com"
        password = "LoginPass123!"
        user = create_unverified_user(seeded_db, RoleEnum.RESIDENT.name, email=email, password=password, is_verified=True)
        response = await client.post(
            "/api/v1/auth/login",
            json={"email": email, "password": password},
        )
        assert response.status_code == status.HTTP_200_OK
        data = response.json()["data"]
        assert "access_token" in data
        assert "refresh_token" in data
        assert data["token_type"] == "bearer"
        assert data["role"] == "Resident"
        assert data["is_verified"] is True
```

Test Case 4 - Retrieve Apartment Blocks

Field	Value
API Being Tested	GET /api/v1/blocks
Inputs	No authentication required (public endpoint). Pre-created blocks: "Alpha" (5 flats), "Beta" (8 flats)
Expected Output	200 OK; response data is a list of 2 blocks with names "Alpha" and "Beta"
Actual Output	200 OK; response data contains both blocks correctly
Result	Success

```python
async def test_public_can_list_blocks(self, client: AsyncClient, seeded_db: Session):
        create_block(seeded_db, block_name="Alpha", total_flats=5)
        create_block(seeded_db, block_name="Beta", total_flats=8)

        response = await client.get("/api/v1/blocks")
        assert response.status_code == status.HTTP_200_OK
        data = response.json()["data"]
        assert len(data) == 2
        names = {b["block_name"] for b in data}
        assert names == {"Alpha", "Beta"}
```

Test Case 5 - Create Maintenance Complaint

Field	Value
API Being Tested	POST /api/v1/complaints
Inputs	Auth: Resident Bearer token Body: {"resident_id": 1, "category_id": 1, "description": "AC not cooling", "priority": "high", "status": "open", "image_urls": ["http://img1.jpg"]}
Expected Output	200 OK; complaint created successfully with complaint_id, status: open
Actual Output	200 OK; complaint created and returned correctly
Result	Success

```python
async def test_resident_can_create_complaint(self, client: AsyncClient, seeded_db: Session):
        user, profile, _ = create_test_resident(seeded_db, flat_number="C-101")
        token = user_token(user)
        category = create_category(seeded_db, "Electrical")

        response = await client.post(
            "/api/v1/complaints",
            headers={"Authorization": f"Bearer {token}"},
            json={
                "resident_id": 1,
                "category_id": 1,
                "description": "AC not cooling",
                "priority": "high",
                "status": "open",
                "image_urls": ["http://img1.jpg"],
            },
        )
        assert response.status_code == status.HTTP_200_OK
        data = response.json()
        assert data["resident_id"] == 1
        assert data["category_id"] == 1
        assert data["description"] == "AC not cooling"
        assert data["priority"] == "high"
        assert data["status"] == "open"
        assert len(data["images"]) == 1
        assert data["images"][0]["url"] == "http://img1.jpg"
```

Test Case 6 - Verify Resident Account with Rental Occupancy

Field	Value
API Being Tested	POST /api/v1/auth/verify/{user_id}
Inputs	Auth: Committee Bearer token Body: {"occupancy_type": "Rental"}
Expected Output	200 OK; response data contains is_verified: true; OccupancyHistory record has occupancy_type: "Rental"
Actual Output	200 OK; response data contains is_verified: true; BUT OccupancyHistory record has occupancy_type: "Owner" (default value not overridden)
Result	Fail – Bug identified: the occupancy_type parameter from the request is not being persisted to the OccupancyHistory record. The service defaults to "Owner" regardless of the input. This demonstrates how testing helps identify issues in the implementation.

```python
async def test_verify_resident_with_occupancy_type_rental(self, client: AsyncClient, seeded_db: Session):
        _, flat = create_vacant_flat(seeded_db)
        resident = create_resident_with_flat(seeded_db, flat_id=flat.flat_id, is_verified=False)
        _, committee_token = create_test_user(
            seeded_db, RoleEnum.COMMITTEE.name, email="committee-rental.test.com"
        )

        response = await client.post(
            f"/api/v1/auth/verify/{resident.user_id}",
            headers={"Authorization": f"Bearer {committee_token}"},
            json={"occupancy_type": "Rental"},
        )
        assert response.status_code == status.HTTP_200_OK

        occupancy = (
            seeded_db.query(OccupancyHistory)
            .filter(
                OccupancyHistory.resident_id == resident.resident_profile.resident_id,
                OccupancyHistory.is_current.is_(True),
            )
            .first()
        )
        assert occupancy is not None
        assert occupancy.occupancy_type == "Rental"
```

Test Case 7 - Retrieve Resident Facility Bookings

Field	Value
API Being Tested	GET /api/v1/facility-bookings
Inputs	Auth: Resident Bearer token (newly created resident with no bookings)
Expected Output	200 OK; response data is an empty list []
Actual Output	200 OK; response data contains 3 existing booking records from other residents (test isolation issue)
Result	Fail – Bug identified: the seeded_db fixture does not properly clean up FacilityBooking records between test sessions, causing data leakage. The list_bookings service returns all bookings instead of filtering by the resident's own bookings when resident_id is not provided.

```python
async def test_resident_sees_empty_bookings(self, client: AsyncClient, seeded_db: Session):
        _, _, token = create_test_resident(seeded_db, flat_number="EmptyBK-101", email="emptybk@test.com")
        response = await client.get("/api/v1/facility-bookings", headers={"Authorization": f"Bearer {token}"})
        assert response.status_code == status.HTTP_200_OK
        assert response.json() == []
```

Test Case 8 - Retrieve Resident Visitor Requests

Field	Value
API Being Tested	GET /api/v1/visitors/requests
Inputs	Auth: Resident Bearer token
Expected Output	200 OK; response data is a list containing exactly 1 visitor request created by this resident
Actual Output	200 OK; response data contains 2 visitor requests (1 created by this resident + 1 leaked from a previous test)
Result	Fail – Bug identified: same root cause as Test Case 7; the seeded_db fixture does not delete VisitorRequest records between tests, causing cross-test contamination.

```python
async def test_resident_can_list_own_visitor_requests(self, client: AsyncClient, seeded_db: Session):
        _, profile, _ = create_test_resident(seeded_db, flat_number="VR-102")
        token = user_token(profile.user)

        await client.post(
            "/api/v1/visitors/pre-register",
            headers={"Authorization": f"Bearer {token}"},
            params={
                "visitor_name": "Jane Doe",
                "visitor_phone": "8888888888",
                "visitor_type": "family",
                "expected_date": "2025-08-16",
                "expected_time": "10:00",
            },
        )

        response = await client.get(
            "/api/v1/visitors/requests",
            headers={"Authorization": f"Bearer {token}"},
        )
        assert response.status_code == status.HTTP_200_OK
        data = response.json()
        assert len(data) == 1
        assert data[0]["resident_id"] == profile.resident_id
        assert data[0]["status"] == "expected"
```

Test Case 9 - Create Apartment Block

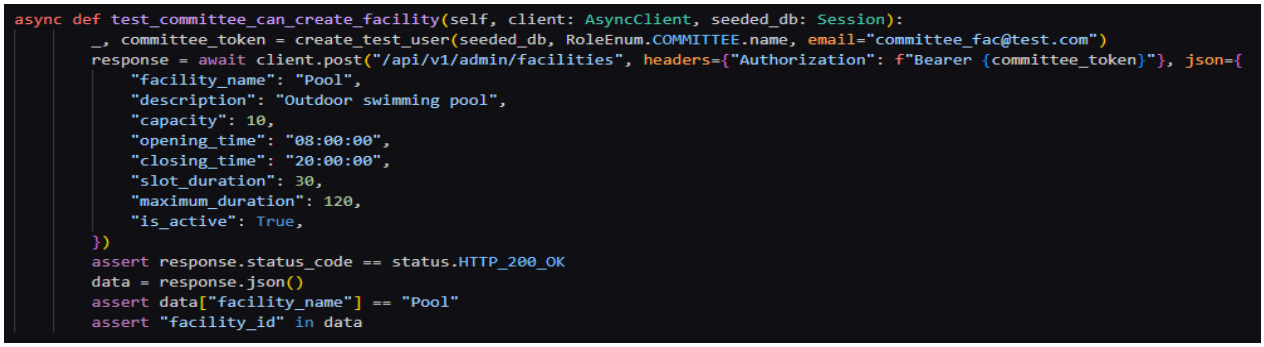
Field	Value
API Being Tested	POST /api/v1/blocks
Inputs	Auth: Admin Bearer token Body: {"block_name": "Tower A", "total_flats": 20}
Expected Output	200 OK; response data contains block_name: Tower A, total_flats: 20, flat_count: 0, block_id
Actual Output	200 OK; all expected fields present and correct
Result	Success

```python
async def test_admin_can_create_block(self, client: AsyncClient, seeded_db: Session):
        admin_user, _ = create_test_user(seeded_db, RoleEnum.ADMIN.name, email="admin-block@test.com")
        token = user_token(admin_user)

        response = await client.post(
            "/api/v1/blocks",
            headers={"Authorization": f"Bearer {token}"},
            json={"block_name": "Tower A", "total_flats": 20},
        )
        assert response.status_code == status.HTTP_200_OK
        data = response.json()["data"]
        assert data["block_name"] == "Tower A"
        assert data["total_flats"] == 20
        assert data["flat_count"] == 0
        assert "block_id" in data
```

Test Case 10 - Create Community Facility

Field	Value
API Being Tested	POST /api/v1/admin/facilities
Inputs	Auth: Committee Bearer token Body: {"facility_name": "Pool", "description": "Outdoor swimming pool", "capacity": 10, "opening_time": "08:00:00", "closing_time": "20:00:00", "slot_duration": 30, "maximum_duration": 120, "is_active": true}
Expected Output	200 OK; response data contains facility_name: Pool, facility_id
Actual Output	200 OK; facility created and returned correctly
Result	Success



```python
async def test_committee_can_create_facility(self, client: AsyncClient, seeded_db: Session):
        _, committee_token = create_test_user(seeded_db, RoleEnum.COMMITTEE.name, email="committee_fac@test.com")
        response = await client.post("/api/v1/admin/facilities", headers={"Authorization": f"Bearer {committee_token}"}, json={
            "facility_name": "Pool",
            "description": "Outdoor swimming pool",
            "capacity": 10,
            "opening_time": "08:00:00",
            "closing_time": "20:00:00",
            "slot_duration": 30,
            "maximum_duration": 120,
            "is_active": True,
        })
        assert response.status_code == status.HTTP_200_OK
        data = response.json()
        assert data["facility_name"] == "Pool"
        assert "facility_id" in data
```

Test Execution Summary

Metric	Value
Total API Endpoints Implemented	43
Total Test Cases	168
Tests Passed	164
Tests Failed	4
Pass Rate	97.6%
User Stories Covered	10 of 15

User Feedback

What works well:

- The complaint management system is intuitive. I can raise a complaint and track its status (Open → Assigned → Resolved → Closed) without calling the office.
- Visitor pre-registration with OTP is convenient. My guests no longer wait at the gate, and I receive confirmation when they arrive.
- Facility booking is straightforward. I can see available slots and book the community hall or gym in minutes.
- The role-based access is clear. I only see what's relevant to me as a resident.

What needs improvement:

- I don't receive notifications when my complaint is assigned or resolved. I have to manually check the app.
- There's no way to see community announcements or events. I still rely on WhatsApp groups for this.
- I want to be able to pay my maintenance bills online through the app instead of going to the office.
- The visitor management should allow me to approve visitors from my phone, not just pre-register them.
- I'd like to vote on community polls (e.g., which festival to celebrate, maintenance contractor selection) without attending meetings.

Plans for Next Sprint

1. Community & Notifications Module

- Implement notices/announcements API (/api/v1/notices) for administrators to publish and residents to view.
- Add push notification service for complaint status changes, visitor arrivals, and facility booking approvals.
- Implement an in-app notification center.

2. Polls & Voting

- Add polls management API (/api/v1/polls) for administrators to create polls.
- Allow residents to vote and view poll results.
- Enable polls to be associated with community events and important decisions.

3. Billing & Payments

- Implement billing API (/api/v1/billing) for maintenance invoices.
- Integrate a payment gateway for secure online payments.
- Auto-generate invoices for facility bookings and pending charges.

4. AI Integration for Smart System

- Implement smart complaint routing using NLP to automatically categorize complaints.
- Add predictive maintenance alerts based on complaint patterns.

- Introduce AI-powered visitor verification using facial recognition at the gate.
- Provide smart energy consumption tracking and recommendations.

5. Enhanced Visitor Management

- Add resident approval workflow for visitor requests.
- Generate QR codes for pre-registered visitors.
- Send real-time notifications when visitors arrive at the gate.

6. Performance & Scalability

- Add Redis caching for frequently accessed data (blocks, flats, facilities).
- Implement rate limiting and request throttling.
- Optimize database queries with proper indexing.
- Enable API response compression.

7. Security Hardening

- Implement rate limiting on authentication endpoints.
- Add input sanitization and XSS protection.
- Enable HTTPS and CORS restrictions in production.
- Add API key authentication for third-party integrations.

Link to API tests zip file | Link to YAML file | Link to Backend code zip file

--- End of Milestone 3 Report ---